

Capital University

Faculty of Computing & Artificial Intelligence

CS316 — Algorithms | Spring 2025–2026

2- Triangle Project Documentation

2- Triangle

A zero-indexed array A consisting of N integers is given. A triplet (P, Q, R) is *triangular* if $0 \leq P < Q < R < N$ and:

1. $A[P] + A[Q] > A[R]$,
2. $A[Q] + A[R] > A[P]$,
3. $A[R] + A[P] > A[Q]$.

For example, consider array A such that:

$A[0] = 10$ $A[1] = 2$ $A[2] = 5$ $A[3] = 1$ $A[4] = 8$ $A[5] = 20$

Triplet $(0, 2, 4)$ is triangular.

Write a function that, given a zero-indexed array A consisting of N integers, returns 1 if there exists a triangular triplet for this array and returns 0 otherwise.

Example 1:

Input: $\text{nums} = [10, 50, 1]$

Output: 0

Example 2:

Input: $\text{nums} = [10, 2, 5, 1, 8, 20]$

Output: 1

=> Non-Recursive Pseudo code :

Algorithm Triangle(A, N)

FOR i $\leftarrow$ 1 TO N - 1 DO

 Key $\leftarrow$ A[i]
 J $\leftarrow$ i - 1

 WHILE (j >= 0) AND (A[j] > key) DO
 A[j + 1] $\leftarrow$ A[j]
 j $\leftarrow$ j - 1

 A[j + 1] $\leftarrow$ key

FOR i $\leftarrow$ 0 TO N - 3 DO

 IF (A[i] + A[i + 1] > A[i + 2]) THEN
 RETURN 1

RETURN 0

=> Non-Recursive Implementation:

```
1  # Non-Recursive
2  def triangle(nums):
3
4      n = len(nums)
5
6      for i in range(1, n):
7
8          key = nums[i]
9          j = i - 1
10
11         while j >= 0 and nums[j] > key:
12             nums[j + 1] = nums[j]
13             j -= 1
14
15         nums[j + 1] = key
16
17     for i in range(n - 2):
18
19         if nums[i] + nums[i + 1] > nums[i + 2]:
20             return 1
21
22     return 0
23
```

=> Non-Recursive Analysis & complexity (steps):

insertion Sort Complexity

The algorithm first sorts the array using **Insertion Sort**.

Best Case

The best case occurs when the array is already sorted.

In this case, the inner while loop executes only one comparison for each iteration without shifting elements.

Therefore, the running time is:

$$T(n) = O(n)$$

Worst Case

The worst case occurs when the array is sorted in reverse order.

In this situation, every element must be compared and shifted to its correct position.

The total number of comparisons is:

$$1 + 2 + 3 + \dots + (n - 1) = \frac{n(n - 1)}{2}$$

This gives a quadratic growth rate.

Therefore, the time complexity is:

$$O(n^2)$$

Average Case

In the average case, approximately half of the elements are shifted.

Thus, the running time remains:

$$O(n^2)$$

2. Triangle Checking Complexity

After sorting, the algorithm checks every three consecutive elements to determine whether they satisfy the triangle condition.

The loop executes: $(n - 2)$ times.

Since only one comparison is performed in each iteration, the time complexity is: $O(n)$

3. Total Time Complexity

The total complexity of the algorithm is:

$$O(n^2) + O(n)$$

the overall time complexity of the algorithm is:

$$O(n^2)$$

=> Recursive Pseudo code :

Algorithm Merge Sort(A)

IF length(A) > 1 THEN

Divide A into left and right

Merge Sort(left)

Merge Sort(right)

Merge left and right into A

Algorithm CheckTriangle(A, i, N)

IF $i > N - 3$ THEN

RETURN 0

IF $(A[i] + A[i + 1] > A[i + 2])$ THEN

RETURN 1

RETURN CheckTriangle(A, $i + 1$, N)

Algorithm TriangleRecursive(A, N)

MergeSort(A)

RETURN CheckTriangle(A, 0, N)

=> Recursive Implementation:

```
1  # Recursive
2
3  def merge_sort(nums):
4
5      if len(nums) > 1:
6
7          mid = len(nums) // 2
8
9          left = nums[:mid]
10         right = nums[mid:]
11
12         merge_sort(left)
13         merge_sort(right)
14
15         i = j = k = 0
16
17         while i < len(left) and j < len(right):
18
19             if left[i] < right[j]:
20                 nums[k] = left[i]
21                 i += 1
22             else:
23                 nums[k] = right[j]
24                 j += 1
25
26             k += 1
27
28         while i < len(left):
29             nums[k] = left[i]
30             i += 1
31             k += 1
32
33         while j < len(right):
34             nums[k] = right[j]
35             j += 1
36             k += 1
37
38     def check_triangle(nums, i):
39
40         if i > len(nums) - 3:
41             return 0
42
43         if nums[i] + nums[i + 1] > nums[i + 2]:
44             return 1
45
46         return check_triangle(nums, i + 1)
47
48
49
50
51     def triangle_recursive(nums):
52
53         merge_sort(nums)
54
55         return check_triangle(nums, 0)
56
57
```

=> Recursive Analysis & complexity (steps):

=> Master method:

$$T(n) = aT(n/b) + f(n)$$

-> Merge sort:

$$T(n) = 2T(n/2) + O(n)$$

$$a = 2 \quad b = 2 \quad f(n) = n \quad K = 1$$

$$\log_b a = \log_2 2 = 1 \quad \therefore \log_b a = K$$

$$\therefore \text{Case 2} \quad \therefore T(n) = O(n \log n)$$

$$\rightarrow \text{check triangle} \quad T(n) = O(n)$$

$$\therefore \text{Total Complexity} = O(n \log n)$$

=> Comparison&Coplexity_Table :

Aspect	Non-Recursive Approach	Recursive Approach
Idea	Uses a loop to check coonsecutive triplets after sorting	Uses recursion to check triplets one by one
Implementation	Simple for loop	Function calling itself
Time Complexity	$O(n^2)$ (sorting) + $O(n)$ (loop) = $O(n^2)$	$O(n \log n)$ (sorting) + $O(n)$ (recursion) = $O(n \log n)$
Space Complexity	$O(1)$ (no extra memory)	$O(n)$ (extra arrays + recursion stack)
Performance	Better for small inputs	Better for large inputs